# LAB 11.5 - FLUTTER TEST SUITE SUMMARY

# REPORT

Course: PRM393 - Mobile Programming (Module 11: Testing & Debugging in Flutter)
Student Name: Hoang Cong Huy | Student ID: HE186488 | Application: Taskly App
Date: November 2026 | Environment: Flutter 3.47.4 • Dart 3.13.3

## 1. Executive Summary & Test Statistics

This document consolidates and summarizes the complete automated testing suite developed for the Taskly Flutter application in accordance with the Module 11 specifications. All test suites were run using the Flutter testing framework (

lutter test).

- **Total Tests Executed:** 14 tests
- **Passed Tests:** 14 tests (100% Pass Rate)
- **Failed / Skipped Tests:** 0 tests
- **Execution Time:** 1.8 seconds
- **Testing Types Included:** Unit Tests, Widget Tests, Navigation Tests, Integration Tests
-

## 2. Complete Test Inventory & Execution Status

| ID | Test Suite / File | Test Description | Category | Status |
|---|---|---|---|---|
| 1 | task_model_test.dart | Default completed value should be false (AAA) | Unit | **PASSED** |
| 2 | task_model_test.dart | toggle() switches completed true <-> false (AAA) | Unit | **PASSED** |
| 3 | task_model_test.dart | copyWith() updates fields and preserves immutability | Unit | **PASSED** |
| 4 | task_repository_test.dart | addTask() inserts task into repository (AAA) | Unit | **PASSED** |
| 5 | task_repository_test.dart | deleteTask() removes task by id (AAA) | Unit | **PASSED** |
| 6 | task_repository_test.dart | updateTask() updates existing task data (AAA) | Unit | **PASSED** |
| 7 | task_repository_test.dart | getTaskById() finds task or returns null | Unit | **PASSED** |
| 8 | task_repository_test.dart | clear() purges all tasks from repository | Unit | **PASSED** |
| 9 | task_list_widget_test.dart | Displays empty state: 'No tasks yet. Add one!' | Widget | **PASSED** |
| 10 | task_list_widget_test.dart | Add Task: enter text -> tap add -> verify UI | Widget | **PASSED** |
| 11 | task_list_widget_test.dart | Multiple Tasks: add two tasks -> both visible | Widget | **PASSED** |
| 12 | task_navigation_test.dart | Tap seeded task -> verify 'Task Detail' & detailTitleField | Navigation | **PASSED** |
| 13 | task_integration_test.dart (widget) | Full flow: Add -> Open detail -> Edit -> Save -> Verify list | Integration | **PASSED** |
| 14 | task_integration_test.dart (integration) | End-to-End lifecycle of a Task across screens | Integration | **PASSED** |

## 3. Behaviors Validated

- **Domain Logic & AAA Pattern:** Arranged, acted, and asserted state transformations in pure Dart for Task model initialization, default completion flags, toggling, and copyWith operations.
- **Repository Operations:** Validated thread-safe state mutations (insertion, deletion, updating, query by ID, and bulk clearing) on TaskRepository.
- **UI Component & Reactive Rendering:** Asserted presence of empty state messages, verified text entry into TextEditingController, validated tap events via WidgetTester, and ensured dynamic UI updates via ListenableBuilder.
- **Screen Transition & Navigation Lifecycle:** Validated route pushing via Navigator.push, verified pumpAndSettle() animations, and checked parameter passing to TaskDetailScreen with exact widget keys (detailTitleField).
- **End-to-End Task Lifecycle:** Simulated realistic user journeys: creating an original task, opening its details, editing the title, persisting via Save, returning to the list, and asserting full UI data consistency.

## 4. Test Suite Structure

The test repository conforms strictly to the Lab 11 directory hierarchy:

```
test/
 ├── unit/              [Pure Dart logic: Task model & repository]
 │    ├── task_model_test.dart
 │    └── task_repository_test.dart
 ├── widget/            [Widget & Navigation tests]
 │    ├── task_list_widget_test.dart
 │    ├── task_navigation_test.dart
 │    └── task_integration_test.dart
 └── integration/       [Consolidated E2E Integration tests]
      └── task_integration_test.dart
```

## 5. Known Limitations & Recommendations

- **In-Memory Storage:** The current repository persists tasks in-memory. Terminating the application resets state. Recommendation: Integrate SQLite (sqflite) or shared_preferences for persistent local storage in future iterations.
- **Headless WidgetTester Environment:** Integration tests currently execute within Flutter's simulated headless environment. Recommendation: Deploy flutter_driver or native integration_test drivers on physical Android and iOS hardware for platform-specific gesture and keyboard testing.
- **Remote Synchronization:** The app does not yet communicate with an external REST/GraphQL API. Recommendation: Introduce mock HTTP clients (e.g. Mockito or mock_web_server) when implementing network synchronization in subsequent modules.
-